

4. naloga
Integracija dinamičnih sistemov v času

Tadej Lozej 28242023

10. december 2025

Praktikum strojnega učenja v fiziki

Predavatelj: red. prof. dr. Borut Paul Kerševan

Asistent: Uroš Perkan

Kazalo

1	Uvod	1
2	Lorentzov '63 sistem	1
2.1	Napovedovanje z RNN mrežo	2
2.2	Napoved z uporabo Luongovega mehanizma pozornosti	4
2.3	Iskanje optimalnih hiperparametrov	5
2.4	Perturbacije modela	6

1 Uvod

2 Lorentzov '63 sistem

Edward Lorenz je ameriški matematik in meterolog, ki je študiral hidrodinamske sisteme. Je eden začetnikov teorije kaosa. Kaotični sistemi so sistemi katerih rešitev se drastično spremenijo pri majhni spremembi začetnih pogojev.

Dinamični sistem, ki ga je študiral Lorenz je poenostavljen opis konvekcije med dvema razsežnima ploskvama z različnima temperaturama. Predpostavimo dvodimenzionalen tok, ki ga opišemo z Navier-Stokesovo enačbo

$$\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} = \frac{1}{\rho} \nabla p + \nu \nabla^2 \vec{v} - g \hat{k} \approx -\frac{1}{\rho_0} \nabla p + \nu \nabla^2 \vec{v} + g \beta (T - T_0) \hat{k}, \quad (1)$$

kjer je g gravitacijski pospešek, ρ gostota zraka, p tlak v tekočini in $\nu = \mu/\rho$ koeficient kinematične viskoznosti.

Prenos toplote pa poteka s konvekcijo in difuzijo

$$\frac{\partial T}{\partial t} + \vec{v} \cdot \nabla T = \kappa \nabla^2 T, \quad (2)$$

kjer je $\kappa = \lambda/(\rho c_p)$ konstanta toplotne difuzije. V stacionarnem stanju je temperaturni profil linearen. Med zgornjo hlajeno ploščo pri temperaturi T_{cold} ter spodnjo ogrevano ploščo T_{hot} , ki sta za H narazen bi bil $T_c(z) = T_{hot} - \Gamma z$, kjer je $\Gamma = (T_{hot} - T_{cold})/H$. Enačbo 2 lahko prepišemo za temperaturne odmake od linearnega profila $\theta(x, y, z) = T(x, y, z) - T_c(z)$ in dobimo

$$\frac{\partial \theta}{\partial t} - w \Gamma + \vec{v} \cdot \nabla \theta = \kappa \nabla^2 \theta. \quad (3)$$

S tem prepisom poenostavimo robne pogoje enačbe

$$\vec{v}(x, y, z) = 0 \quad \text{za} \quad z = 0, H \quad (4)$$

$$\theta(x, y, z) = 0 \quad \text{za} \quad z = 0, H. \quad (5)$$

Če upoštevamo, da je tok nestisljiv ($\nabla \cdot \vec{v} = 0$) lahko vektorsko polje hitrosti zapišemo s tokovno funkcijo, tako da velja $\vec{v} = \nabla \times \vec{\psi}$, kjer je $\vec{\psi} = (0, \psi(x, z), 0)^T$. Enačbo 3 lahko zapišemo s tokovno funkcijo

$$\frac{\partial \theta}{\partial t} - \frac{\partial \psi}{\partial z} \frac{\partial \theta}{\partial x} + \frac{\partial \psi}{\partial x} \frac{\partial \theta}{\partial z} = \kappa \nabla^2 \theta + \Gamma \frac{\partial \theta}{\partial x}. \quad (6)$$

Rešitev lahko poiščemo z 2D Fourierovim razvojem funkcij ψ in θ . V tem primeru dobimo neskončen sistem sklopljenih diferencialnih enačb. Lorentz je reševal takšen sistem a je vzel le minimalno potrebnih koeficientov za nelinearen sistem. V brezdimenzijskih količinah se Lorentzov '63 reduciran sistem enačb glasi

$$\frac{dX}{d\tau} = \sigma(Y - X) \quad (7)$$

$$\frac{dY}{d\tau} = X(r - Z) - Y \quad (8)$$

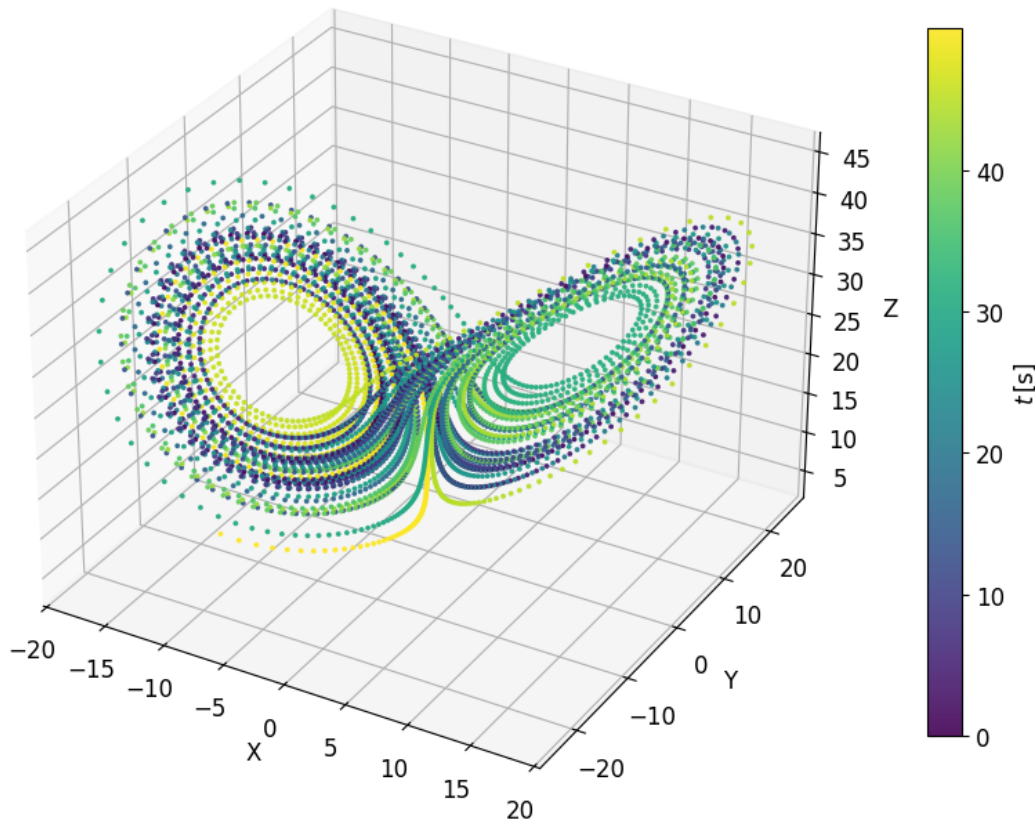
$$\frac{dZ}{d\tau} = XY - bZ, \quad (9)$$

kjer je parameter $\sigma = \nu/\kappa$ Prandtlovo število, $r = R/R_c$ je Rayleighovo število in $b = 4/(1 + a^2)$ je pa odvisen od vertikalne razsežnosti plasti. Za vrednosti parametrov $\sigma = 10$, $b = 8/3$ in $r = 28$ je sistem kaotičen.

Rešitve Lorentzovega sistema čez nekaj časa preidejo na nek del prostora, ki mu rečemo atraktor in tam tudi ostane. Rešitev Lorentzovega sistema za naključni začetni pogoj lahko vidimo na sliki 1. Pridobljena je bila z numeričnim integratorjem Runge-Kutta četrte stopnje. Rešitev se nepredvidljivo giblje po atraktorju.

Sistem je podobno kot atmosfera nelinearen ter kaotičen. Glavna ideja uporabe nevronske mreže v meteorologiji je pohitritev ter paraleliziranje napovedi.

Rešitev Lorentzovega sistema enačb



Slika 1: Rešitev Lorentzovega sistema.

2.1 Napovedovanje z RNN mrežo

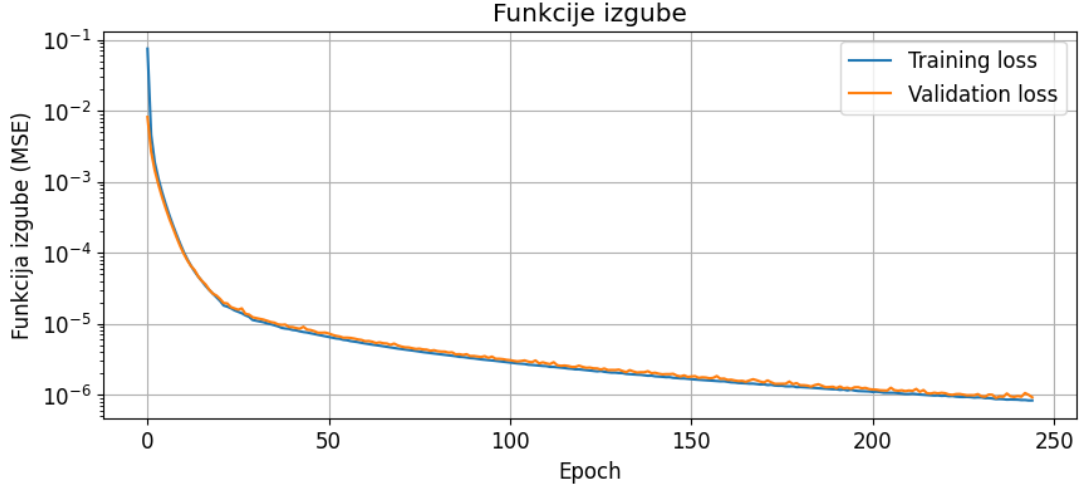
Rekurentne nevronske mreže (RNN) se uporabljajo za procesiranje časovnih vrst. Uporabne so predvsem za napovedovanje prihodnosti časovnih vrst, saj so te ponavadi strogo determinirane s prejšnjim stanjem. Lahko si jo predstavljamo podobno kot Kalmanov filter.

Rekurentna nevronska mreža ima v sebi skrito stanje, ki se rekurentno posodablja po vsakem novem prebranem vhodnem vektorju. Skrito stanje deluje kot spomin nevronske mreže, saj vsako novo stanje vsebuje informacijo o preteklih vhodnih podatkih zahvaljujoč skritemu stanju. Rekurentno posodabljanje skritih stanj pa vseeno povzroča hitro izgubo informacij o preteklih vhodih, zato pravimo, da imajo RNN kratkoročni spomin.

Učne podatke pripravimo z integracijo Lorentzovega sistema z numeričnim integratorjem Runge-Kutta četrtega reda. Podatke standardno normaliziramo ter sekvenciramo. Vhodi v RNN so sekvence dolžine 5, izhodi so pa dolžine 1. Poleg učne množice seveda pripravimo tudi validacijsko množico.

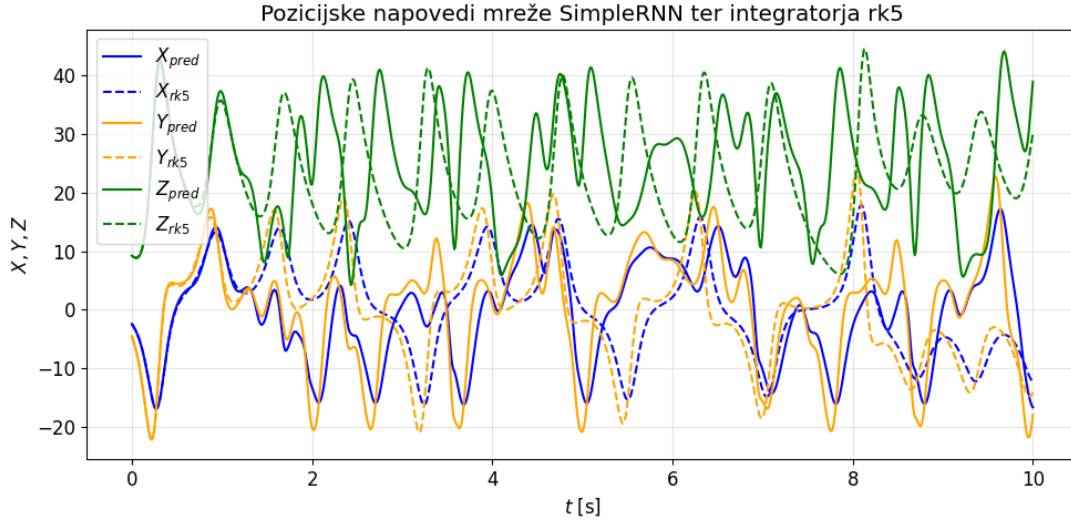
Pri učenju nevronske mreže uporabimo callbacka `EarlyStopping` z potrpežljivostjo 10 ter `ReduceLROnPlateau`. Na sliki 2 lahko vidimo učno (modra) ter validacijsko (oranžno) funkcijo izgube pri učenju RNN v logaritemski skali. Pri učenju smo parameter `batch_size` postavili na 300. Učna funkcija izgube je kot pričakovano zelo gladka. Presenetilo me je kako malo zašumljena je validacijska funkcija izgube. Mreža se je učila skoraj 250 epochov in funkcija izgube je dosegla vrednost okrog 10^{-6} . Funkcija izgube je mean squared error (MSE).

Na sliki 3 lahko s polno črto vidimo eno izmed dvesto napovedi v odvisnosti od časa RNN in s črtkano črto rešitve pridobljene z numeričnim integratorjem. Različne barve prikazujejo različne koordinate. Napoved RNN je pridobljena tako, da mu podamo začetno sekvenco nato pa svoje napovedi uporabi za naslednjo napoved. To si lahko predstavljamo kot Kalmanov filter brez izostritve zaradi novih meritev. RNN je seveda v tovrstnih napovedih precej boljša kot Kalmanov filter, a se mi zdi dober za analogijo. Kot pričakovano



Slika 2: Učna (modra) funkcija izgube ter validacijska (oranžna) funkcija izgube pri učenju RNN.

lahko vidimo, da se napovedi s časom vedno bolj odmikajo od vrednosti dobljene z integratorjem, ki jo bomo vzeli za resnično vrednost. To je smiselno, saj je integrator precej bolj natančna metoda, če sistem diferencialnih enačb poznamo. Prav tako smo mrežo natrenirali na podatkih dobljenih z integratorjem.

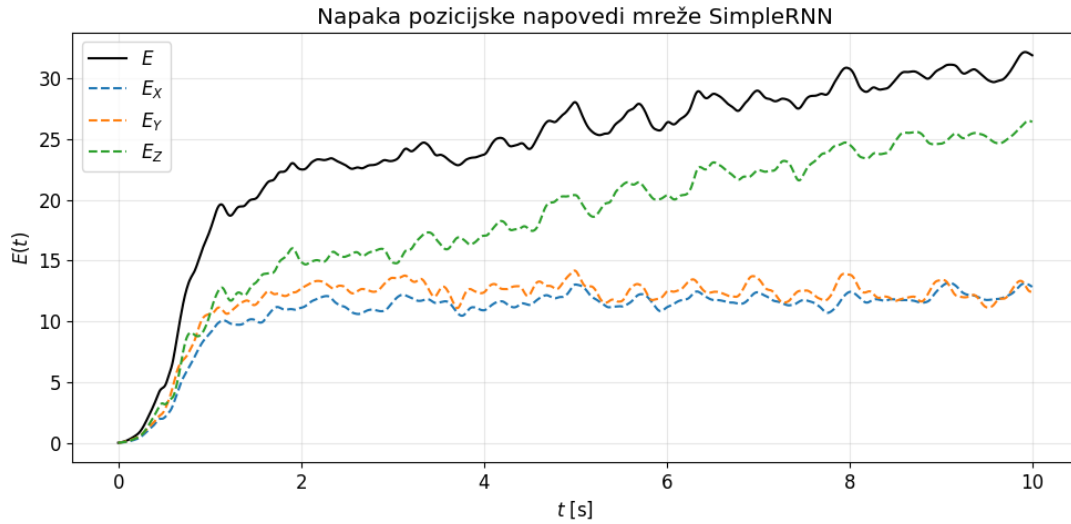


Slika 3: Napovedi Lorentzovega sistema z RNN (polne črte) ter vrednosti pridobljene z integratorjem (črtkane črte). Različne koordinate so prikazane z različnimi barvami.

Napako napovedi definiramo kot

$$E(t) = \sqrt{E_X^2 + E_Y^2 + E_Z^2}; \quad E_\alpha = \sqrt{\frac{1}{N} \sum_{i=1}^N (\alpha_{pred} - \alpha_{rk5})^2} \quad (10)$$

kjer je E_α koren iz povprečne razlike kvadrata med napovedano vrednostjo ter pravo vrednostjo koordinate. Povprečimo čez 200 različnih napovedi. Na sliki 4 lahko vidimo kako so napake posameznih koordinat ter skupna napaka odvisne od časa. Kot pričakovano vidimo, da s časom napaka narašča. Zanimivo se mi zdi, da se napake v smereh X ter Y ustalijo pri neki vrednosti, napaka v smeri Z pa ne. Mogoče pri več primerih dobimo lezenje v tej smeri.

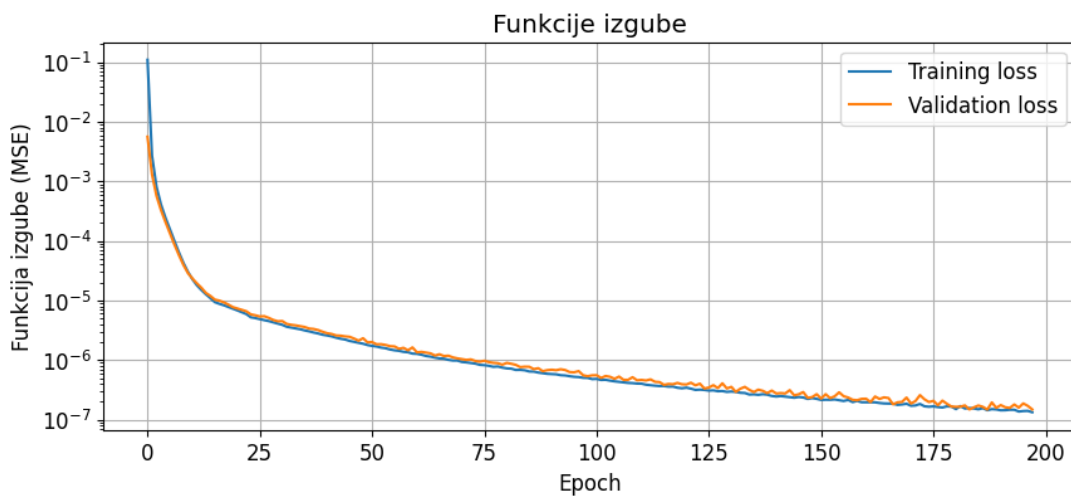


Slika 4: Napaka napovedi RN v odvisnosti od časa definirana z enačbo 10. Z črtkanimi črtami različnih barv so prikazane napake posameznih koordinat.

2.2 Napoved z uporabo Luongovega mehanizma pozornosti

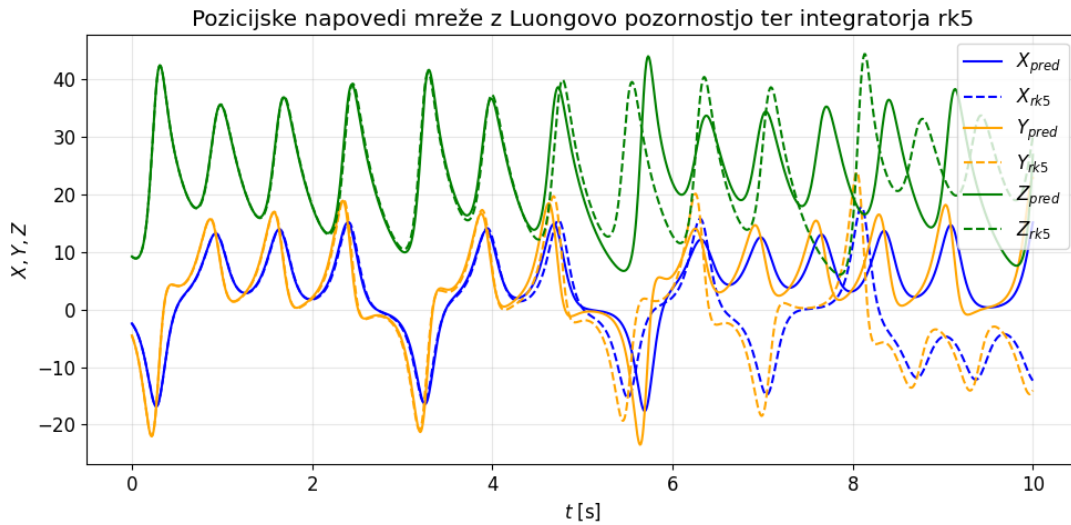
Težavo kratkoročnega spomina lahko rešimo z uporabo Luongovega mehanizma pozornosti. Ta deluje za vse tipe nevronske mreže, ki so sestavljene iz kodirnika ter dekodirnika. Kodirnik časovno vrsto vhodnih vektorjev preslika v skrita stanja. Običajna RNN bi ta skrita stanja z gosto povezano nevronske mreže preslikala v napoved. Glavna ideja Luongovega mehanizma pozornosti se nahaja pri vektorju konteksta. Mreža ta vektor konstruira tako, da primerja tarčno skrito stanje z vsemi skritimi stanji. Vektor konteksta je z utežmi poln vektor. Te uteži določajo linearno kombinacijo kodirnih skritih stanj. Tega nato združimo s tarčnim skritim stanjem in vse skupaj preslikamo v novo skrito stanje, ki je obogateno s kontekstom. To skrito stanje ima osvežen spomin o oddaljenih skritih stanjih.

Učenje mreže z Luongovo pozornostjo poteka na enak način kot učenje RNN. Tudi callbacke ter učne parametre ohranimo enake za boljšo primerjavo modelov. Na sliki 5 lahko vidimo učno (modro) ter validacijsko (oranžno) funkcijo izgube. V primerjavi z funkcijami izgube na sliki 2 lahko vidimo, da se je model z implementirano Luongovo pozornostjo v manj epochah bolje naučil. Izguba tega modela je v manj kot 200 epochah dosegla vrednost okrog 10^{-7} . To pomeni, da se je mreža v manj ciklih učenja bolje naučila. To je dober pokazatelj, da je ta model boljši.



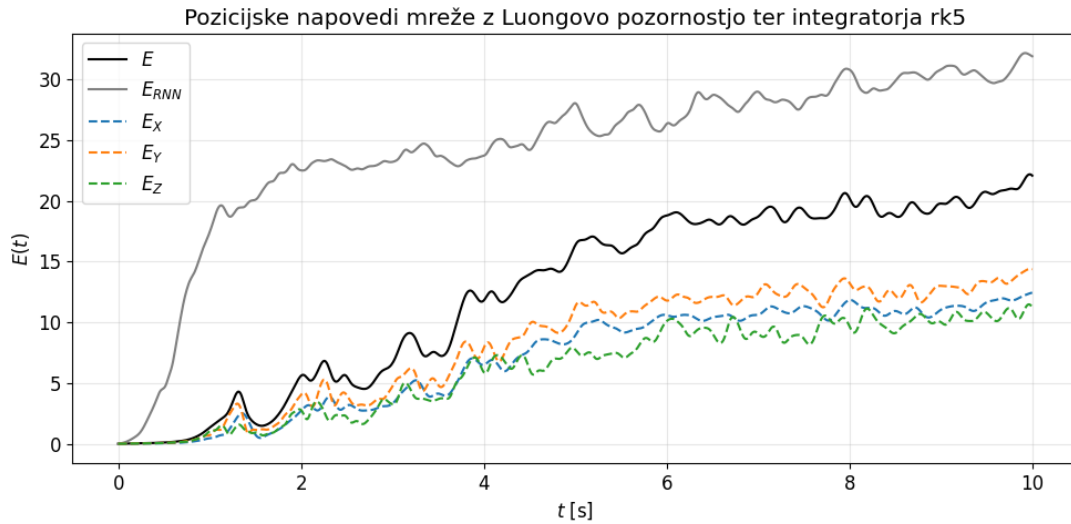
Slika 5: Učna (modra) funkcija izgube ter validacijska (oranžna) funkcija izgube pri učenju mreže z Luongovo pozornostjo.

Na sliki 6 lahko vidimo primer napovedi iz enakih začetnik pogojev kot pri RNN napovedi na sliki 3. S polno črto vidimo napovedi mreže z Luongovo pozornostjo, s črtkano črto pa realne vrednosti. Z različnimi črtami so prikazane različne koordinate. Vidimo lahko, da je v tem primeru napoved tega modela boljša kot napoved RNN modela. Dlje časa napoved ostane precej natančna.



Slika 6: Napovedi Lorentzovega sistema z mrežo z Luongovo pozornostjo (polne črte) ter vrednosti pridobljene z integratorjem (črtkane črte). Različne koordinate so prikazane z različnimi barvami.

Napako napovedi mreže z Luongovo pozornostjo v odvisnosti od časa lahko vidimo na sliki 7. Napaka je izračunana po enačbi 10. S črno je prikazana napaka s sivo je pa prikazana napaka v odvisnosti od časa RNN mreže za primerjavo. Lahko vidimo, da je napoved mreže z Luongovo pozornostjo precej natančnejša. S črtkanimi črtami različnih barv lahko vidimo tudi napake posameznih koordinat. Vse koordinate k napaki prispevajo približno enako.



Slika 7: Napaka napovedi mreže z Luongovo pozornostjo v odvisnosti od časa definirana z enačbo 10. Z črtkanimi črtami različnih barv so prikazane napake posameznih koordinat. S sivo barvo je prikazana napaka napovedi RNN mreže za primerjavo.

2.3 Iskanje optimalnih hiperparametrov

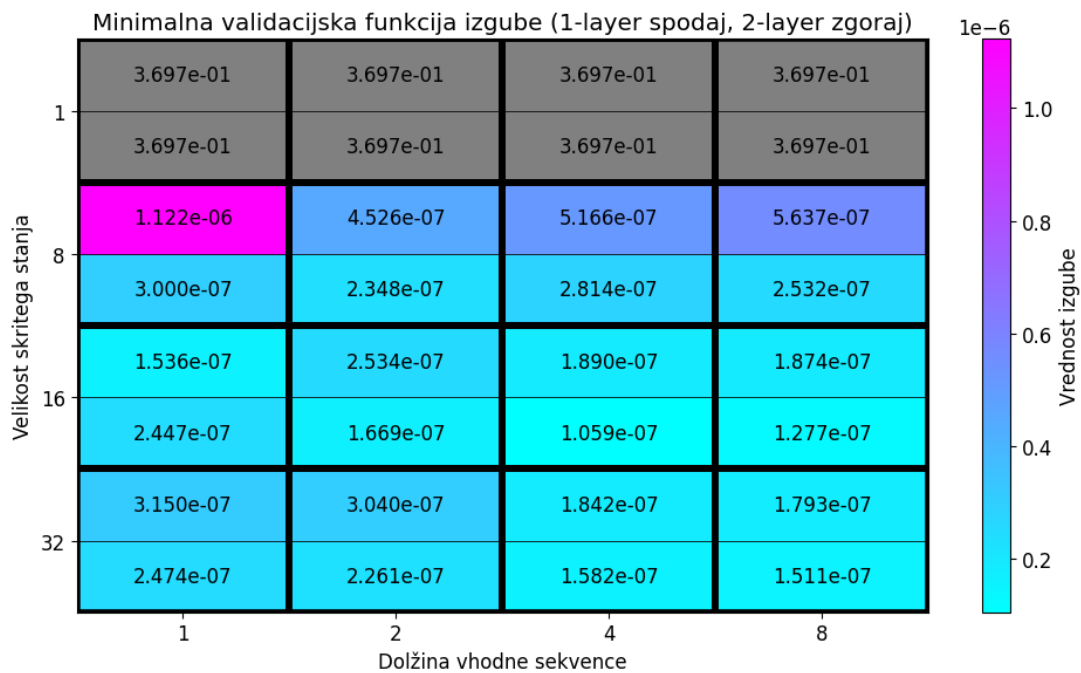
Optimalne parametre bomo iskali pri modelih z implementirano Luongovo pozornostjo. Za modele z eno in dvema plastema v kodirniku ter dekodirniku bomo spreminjali dolžino vhodne sekvence ter velikost

skritega stanja. Načrtovanje stopnje učenja in število epoch se mi je zdelo brezpredmetno spreminjati zaradi uporabljenih callbackov. Velikost batch-a in velikost učne množice pa nisem spreminjal.

Na sliki 8 lahko vidimo najmanjše vrednosti funkcije izgube na validacijski množici različnih modelov. Na y osi so modeli razporejeni po velikosti skritega stanja. Na x osi so modeli razporejeni po dolžini vhodne sekvence. Vsak kvadrateg je razdeljen še na dva dela. V zgornjih delih so zapisani rezultati modelov z dvoslojnim kodirnikom ter dekodirnikom. V spodnjih delih so zapisani rezultati modelov z enoslojnim kodirnikom ter dekodirnikom.

V prvi vrstici lahko vidimo, da z velikostjo skritega stanja 1 ne dosežemo prav veliko. Zanimivo je, da je vrednost izgube na prve tri decimalke enaka v vseh primerih. V poznejših delimalkah se razlikuje. Vidimo tudi, da so v splošnem modeli z enoslojnim kodirnikom ter dekodirnikom bolj napovedali prihodnost kot modeli z dvoslojnima. Napoved je boljša v primeru, ko imamo velikost skritega stanja 16 namesto 8. A se bolj kot ne poslabša, če dimenzijo povečamo na 32. Prav tako v splošnem vidimo izboljšavo napovedi, če povečamo dolžino vhodne sekvence.

Glede na rezultate je najboljši model model z enoslojnim kodirnikom ter dekodirnikom ter z velikostjo skritega stanja 16 in dolžino vhodne sekvence 4. Ta je celo boljši od prejšnjega modela na sliki 5, ki je imel dolžino vhodne sekvence 5.



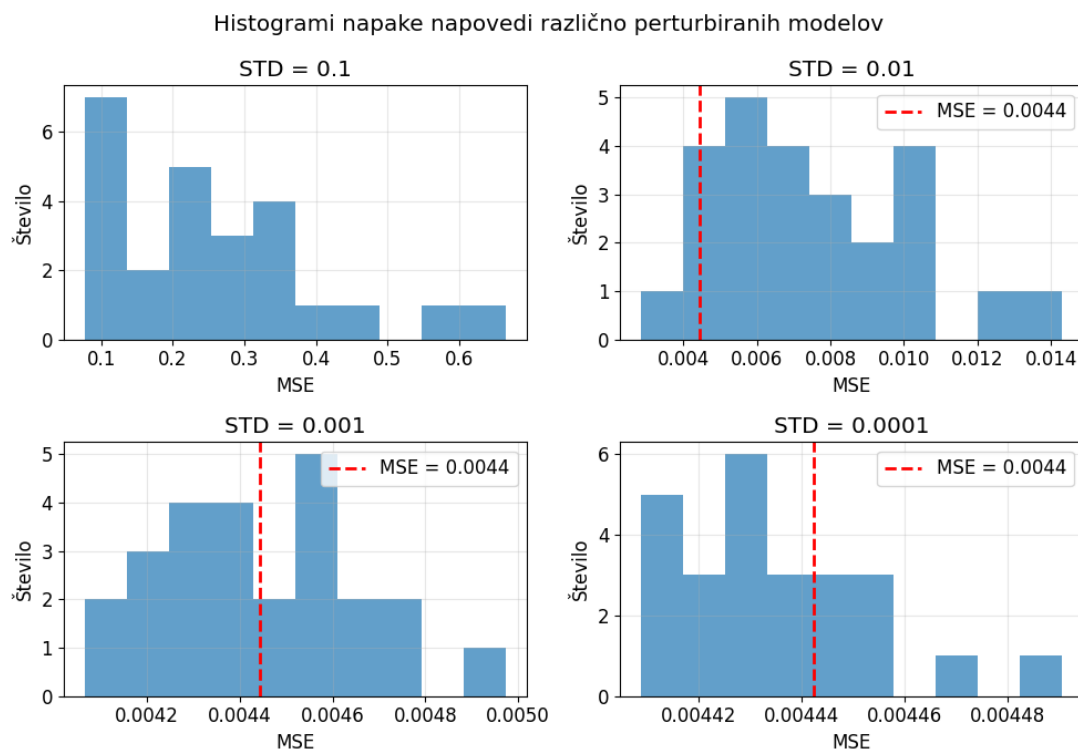
Slika 8: Minimalne vrednosti validacijske funkcije izgube za modele z različno velikostjo skritega stanja ter dolžino vhodne sekvence. V vseh različnih kombinacijah so prikazani rezultati modelov s enoslojnim (spodnji del kvadratkov) ter dvoslojnim (zgornji del kvadratkov) kodirnikom ter dekodirnikom.

2.4 Perturbacije modela

Uteži naučenega modela najverjetneje niso optimalne. Zelo verjetno se nahajamo blizu minimuma vendar ne nujno prav v njem. Zato lahko naključno perturbiramo uteži modela z Gaussovskim šumom in pogledamo, če slučajno pridemo bližje minimumu in imamo posledično boljšo napoved. Prav tako lahko z ansamblom naključno perturbiranih modelov napovemo v kakšnem tipu minimuma smo. Lahko smo v zelo ozkem ali širokem minimumu.

Na sliki 9 lahko vidimo histograme napovedi perturbiranih modelov. Perturbirali smo uteži najboljšega modela, ki smo ga dobili v podpoglavju 2.3. Uteži smo perturbirali z Gaussovskim šumom različnih standardnih deviacij (STD). Uteži smo perturbirali s štirimi različnimi STD-ji. Natančnost napovedi smo pridobili s funkcijo `model.evaluate`. Evalvirali smo na validacijski množici. Z rdečo črtkano črto je prikazana evalvacija originalnega modela. Vidimo, da če perturbiramo uteži z standardno deviacijo 0.1 dobimo zelo slabe

modele. To je očitno prevelika perturbacija. Pri manjših STD-jih pa dobimo tudi boljše modele od originalnega modela. Pri STD 0.01 jih je vseeno precej slabših. Pri manjših standardnih deviacijah pa so evalvacije modelov lepo razporejene okrog originalnega modela.



Slika 9: Napaka napovedi mreže z Luongovo pozornostjo v odvisnosti od časa definirana z enačbo 10. Z črtkanimi črtami različnih barv so prikazane napake posameznih koordinat. S sivo barvo je prikazana napaka napovedi RNN mreže za primerjavo.